

INTRODUCCIÓN A LOS SISTEMAS DISTRIBUIDOS / REDES
(75.43/75.33/95.60/TA048) CURSO 01 ALVAREZ HAMELIN (MARTES)

Trabajo Práctico 1

File Transfer

5 de mayo de 2026

- Giuseppe Mancinelli
112066
- Noel Alejandro Escobar Barreto
111151
- Joani Alejandro Pranteda
108303
- Gonzalo Laos Pinto
109967
- Luis Trebejo
109988

1. Introducción

El presente trabajo práctico aborda el diseño e implementación de una aplicación de red con arquitectura cliente-servidor orientada a la transferencia de archivos. El objetivo principal se centra en comprender el modelo de servicio de la capa de transporte y construir un mecanismo de transferencia de datos confiable (RDT) sobre un protocolo inherentemente no confiable y no orientado a la conexión como UDP.

Para satisfacer los requerimientos del sistema, se desarrollaron dos estrategias de recuperación de errores con funcionamientos sumamente distintos en su manejo del estado de la red: el protocolo Stop & Wait y el protocolo Selective Repeat. La solución fue diseñada para operar de manera concurrente en una topología simulada mediante la herramienta Mininet. El desafío central consistió en garantizar la entrega íntegra de archivos de hasta 10 MB en un lapso máximo de 2 minutos, contemplando un flujo de errores y manteniendo la resiliencia frente a una tasa de pérdida de paquetes del 10% sobre los enlaces.

2. Hipótesis y suposiciones realizadas

- **Aislamiento de concurrencia mediante archivos temporales:** Para cumplir con el procesamiento concurrente de múltiples clientes, se asume una política de concurrencia delegada al sistema operativo. Se descartó el uso de bloqueos (locks/mutex) sobre archivos o el directorio. Se determinó que cada operación de subida genera de forma transparente un descriptor de archivo temporal y único. Las colisiones en el nombrado se resuelven bajo el paradigma "Last-Write-Wins", aplicando un renombrado atómico sobre el archivo únicamente cuando se valida la integridad total de la transferencia (transferencia completa). De esta manera, se previenen los interbloqueos (deadlocks) y se permite que operaciones simultáneas sobre el mismo archivo no corrompan los flujos de bytes.
- **Negociación temprana de estado (Handshake Inicial):** El protocolo exige que la transferencia de datos esté precedida por un paquete de control donde el cliente informa su intención (carga o descarga), el protocolo seleccionado (Stop & Wait o Selective Repeat) y el peso en bytes del archivo a transmitir. Esta suposición permite al hilo despachador del servidor instanciar dinámicamente la clase receptora adecuada y habilitar una validación de peso máximo de archivo. Si el peso declarado excede el límite del servidor, la conexión es rechazada inmediatamente antes de comprometer memoria o ciclos de I/O, resolviendo limpiamente una condición de error.

3. Implementación

La arquitectura del sistema se basa en un modelo Cliente-Servidor interactuando a través de la interfaz de sockets sobre el protocolo UDP. El lenguaje utilizado fue Python y el desarrollo respeta el estándar PEP8 para garantizar la legibilidad y mantenibilidad del código. Para sortear las limitaciones inherentes a la falta de confiabilidad de UDP, se desarrolló un protocolo de capa de aplicación que orquesta la transferencia mediante una cabecera estandarizada y rutinas de inicialización definidas.

3.1. Diseño del Protocolo de Capa de Aplicación

Se definió una cabecera estándar de 7 bytes que precede a todo datagrama transmitido por la red, estructurada de la siguiente manera:

- **Opcode o Código de Operación (1 Byte):** Define la naturaleza del paquete. Se reservaron valores específicos para diferenciar entre solicitudes como UPLOAD/DOWNLOAD, transferencia de datos regulares, banderas de fin de archivo (FIN), acuses de recibo (ACK) y notificaciones de error del servidor.
- **Número de Secuencia (4 Bytes):** Un entero sin signo (unsigned int) que identifica unívocamente la posición lógica del paquete dentro del flujo de la transferencia.
- **Longitud del Payload (2 Bytes):** Un entero corto sin signo (unsigned short) que determina la cantidad de bytes de datos transmitidos útiles (payload) que subsiguen a la cabecera.

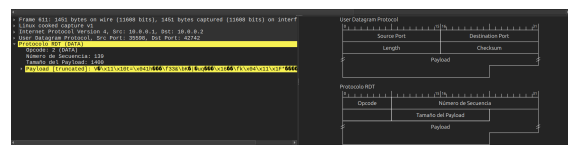


Figura 1: Visualización desde Wireshark de la cabecera de un paquete de datos regulares (DATA)

3.2. Handshake Inicial

El ciclo de vida de una transferencia comienza con una fase de negociación temprana o Handshake. El cliente inicializa la comunicación enviando un datagrama de control cuyo Opcode indica la operación deseada (UPLOAD o DOWNLOAD).

El payload de este datagrama inicial transporta los metadatos críticos de la sesión a comenzar: el protocolo de recuperación de errores seleccionado (Stop & Wait o Selective Repeat, codificado en 1 byte), el tamaño total del archivo (codificado en 8 bytes como un entero largo sin signo) y el nombre del archivo en formato UTF-8.

Esta arquitectura de inicialización le permite al hilo despachador del servidor poder fallar tempranamente. Al recibir el peso del archivo a descargar, el servidor valida si la operación supera el límite configurado previamente en el servidor (por ejemplo, 20 MB). De ser así, aborta el proceso de inmediato y responde con un Opcode de error, satisfaciendo la exigencia de contemplar condiciones de error sin saturar los recursos de disco o memoria. En caso de aceptación, el servidor despacha la conexión instanciando dinámicamente la una clase receptora correspondiente al protocolo RDT solicitado por el cliente, permitiendo la atención concurrente de múltiples clientes con flujos heterogéneos (con distintos protocolos).

3.3. Mecanismos de Transferencia Confiable (RDT)

La transmisión de los bloques de datos quedan establecidos por los protocolos elegidos por el cliente, los cuales garantizan la llegada de la información sobre un canal no confiable (UDP):

- **Stop & Wait:** Este mecanismo opera bajo una lógica de confirmación individual. El emisor envía un único datagrama y detiene la transmisión hasta recibir el ACK correspondiente. Si el temporizador expira antes de la confirmación, se retransmite el bloque. Es un protocolo de alta confiabilidad y baja complejidad, pero con una utilización del canal ineficiente en redes de alta latencia.
- **Selective Repeat:** Implementa la técnica de *pipelining*, permitiendo que múltiples paquetes estén “en vuelo” simultáneamente. El emisor mantiene una ventana de transmisión y el receptor cuenta con un buffer para almacenar paquetes que lleguen fuera de orden. Ante una pérdida, solo se retransmite el paquete específico que falló, optimizando drásticamente el flujo de datos y el ancho de banda.

3.4. Flujos Principales

La decisión principal de diseño fue separar la escucha de nuevas conexiones del proceso de transferencia de datos.

Para gestionar la concurrencia, se implementó un flujo donde un *Dispatcher* actúa como receptor central. Al recibir una solicitud válida, este no procesa la transferencia, sino que instancia un hilo independiente (*Worker*) con un socket propio en un puerto aleatorio. Esta estrategia permite que el servidor mantenga su disponibilidad constante para atender nuevos clientes mientras los hilos secundarios gestionan la lógica pesada de lectura de disco y retransmisiones.

La comunicación se formaliza mediante una jerarquía de clases de datagramas, donde cada paquete contiene un encabezado (*header*) que identifica el tipo de operación (*opcode*). Este flujo estructurado permite que el sistema reaccione de forma determinística ante cualquier evento de red.

3.5. Manejo de Errores

El sistema implementa una gestión de errores en múltiples capas, asegurando que fallos críticos no dejen al servidor o al cliente en estados inconsistentes (como puertos bloqueados o memoria sin liberar).

- **Errores de Aplicación y Protocolo:** Se utiliza una clase especializada, *ErrorDatagram*, para notificar fallos entre los nodos. Por ejemplo, si un cliente solicita un archivo inexistente o de tamaño superior al límite permitido, el servidor no simplemente corta la conexión, sino que intenta informar la causa mediante la función `send_error_reliably`.
- **Timeouts y Control de Flujo:** El manejo de la inactividad y la pérdida de paquetes es el segundo pilar. Se definieron excepciones personalizadas para gestionar el agotamiento de intentos.

Ejemplo de manejo de excepciones en el cliente:

```
1 try:
2     cliente = ClientDownloader(args.host, args.port, args.protocol, logger)
3     cliente.download_file(args.name, args.dest)
4     logger.info("Transfer completed successfully.")
5 except Exception as e:
6     logger.error(f"Critical failure during transfer: {e}")
7 sys.exit(1)
```

Dentro de la lógica de transferencia, el sistema controla la falta de respuesta del par mediante temporizadores:

- **Inactividad:** Si no se reciben datos tras un tiempo prolongado, se lanza: `raise ConnectionError("Sender inactivity timeout")`.
- **Persistencia:** Si un bloque falla repetidamente tras agotar los reintentos configurados: `raise ConnectionError(f"Max retries reached for block seq_num")`.

Finalmente, el sistema cuenta con un Logger configurable que permite distintos niveles de verbosidad (DEBUG, INFO), lo cual fue fundamental durante el desarrollo para trazar el origen de los errores en entornos multihilo.

4. Pruebas

Se realizaron pruebas exhaustivas del protocolo RDT implementado en un entorno controlado usando Mininet. La topología utilizada simula una red con tres nodos (1 servidor y 2 clientes) conectados a un switch central, con una pérdida de paquetes del 5 % en cada enlace (10 % de tasa total en el camino cliente-servidor).

Se implementaron y evaluaron dos estrategias de recuperación de errores: Stop & Wait (SW) y Selective Repeat (SR). Para cada protocolo, se realizaron pruebas de transferencia de archivos en ambas direcciones (UPLOAD y DOWNLOAD).

Las pruebas incluyeron el monitoreo en tiempo real del tráfico de red utilizando Wireshark, con un disector Lua personalizado que permite visualizar los campos de la cabecera del protocolo (Opcode, Número de Secuencia, Tamaño del Payload) y clasificar cada paquete según su tipo de operación (HANDSHAKE, DATA, ACK, ERROR, CLOSE).

Se capturó el comportamiento de ambos protocolos ante condiciones de red adversas, permitiendo verificar la correcta implementación de los mecanismos de retransmisión, la validación de secuencias y la finalización de conexiones. En todos los escenarios expuestos a continuación, el nodo que asume el rol de Servidor opera bajo la dirección IP 10.0.0.1 (h1), mientras que el Cliente se identifica con la dirección IP 10.0.0.2 (h2).

Primeramente, veamos el caso de un UPLOAD utilizando el protocolo de Stop & Wait:

A continuación se presentan las trazas de red obtenidas mediante Wireshark y la salida estándar de los hosts terminales. A través del disector, se ilustran las transiciones de la máquina de estados durante la sincronización inicial (Handshake), la transmisión de datos del archivo y la liberación de recursos (4-way Teardown). Esto permite auditar la evolución de los campos de control en la cabecera (tales como Opcode y Sequence Number) y verificar el determinismo del protocolo frente al entorno de red adverso emulado en Mininet.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.2	10.0.0.1	RDT	69	HANDSHAKE
2	0.000221269	10.0.0.2	10.0.0.1	RDT	51	ACK Seq=0
3	0.000689653	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
4	0.000829139	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
5	0.000941452	10.0.0.2	10.0.0.1	RDT	1451	DATA Seq=1
6	0.000945939	10.0.0.2	10.0.0.1	RDT	1451	DATA Seq=1
7	0.101299384	10.0.0.2	10.0.0.1	RDT	1451	DATA Seq=1
8	0.101331969	10.0.0.2	10.0.0.1	RDT	1451	DATA Seq=1
9	0.101603597	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=1
10	0.101625377	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=1
11	0.101904697	10.0.0.2	10.0.0.1	RDT	1451	DATA Seq=2
12	0.101924117	10.0.0.2	10.0.0.1	RDT	1451	DATA Seq=2
13	0.102183701	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=2

Figura 2: Handshake de Upload

No.	Time	Source	Destination	Protocol	Length	Info
610	3.239996212	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=140
611	3.340259805	10.0.0.2	10.0.0.1	RDT	688	DATA Seq=141
612	3.340273123	10.0.0.2	10.0.0.1	RDT	688	DATA Seq=141
613	3.340426972	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=141
614	3.340431224	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=141
615	3.340552687	10.0.0.2	10.0.0.1	RDT	51	CLOSE
616	3.340556985	10.0.0.2	10.0.0.1	RDT	51	CLOSE
617	3.340641103	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=142
618	3.340656815	10.0.0.1	10.0.0.2	RDT	51	CLOSE
619	3.340658299	10.0.0.1	10.0.0.2	RDT	51	CLOSE
620	3.390822790	10.0.0.2	10.0.0.1	RDT	51	CLOSE
621	3.441095996	10.0.0.2	10.0.0.1	RDT	51	CLOSE
622	3.441122264	10.0.0.2	10.0.0.1	RDT	51	CLOSE
623	3.491445564	10.0.0.2	10.0.0.1	RDT	51	CLOSE
624	3.491455482	10.0.0.2	10.0.0.1	RDT	51	CLOSE
625	3.541025318	10.0.0.2	10.0.0.1	RDT	51	CLOSE
626	3.541039209	10.0.0.2	10.0.0.1	RDT	51	CLOSE
627	3.592454873	10.0.0.2	10.0.0.1	RDT	51	ACK Seq=143
628	3.592473484	10.0.0.2	10.0.0.1	RDT	51	ACK Seq=143

Figura 3: 4-way Teardown

Salida de la consola de cada host:

Host 1	Host 2
--------	--------

Figura 4: Vista de las consolas de los hosts

El análisis de estas capturas demuestra que el protocolo logra mantener la transmisión a pesar de la tasa de pérdida del 10 % en el canal. Particularmente en la fase de 4-way Teardown, se evidencia una correcta gestión de los temporizadores de retransmisión para los paquetes de control, mitigando escenarios de conexiones semi-abiertas.

Ahora veamos el caso del Download con Selective Repeat:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.2	10.0.0.1	RDT	62	DOWNLOAD
2	0.000165970	10.0.0.2	10.0.0.1	RDT	62	DOWNLOAD
3	0.000688679	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
4	0.000759280	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
5	0.051264723	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
6	0.051219439	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
7	0.101519870	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
8	0.191537810	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
9	0.201944697	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
10	0.201957804	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=0
11	0.257505583	10.0.0.1	10.0.0.2	RDT	1451	DATA Seq=1
12	0.257522972	10.0.0.1	10.0.0.2	RDT	1451	DATA Seq=1
13	0.257570306	10.0.0.1	10.0.0.2	RDT	1451	DATA Seq=2
14	0.257573763	10.0.0.1	10.0.0.2	RDT	1451	DATA Seq=2
15	0.257599059	10.0.0.1	10.0.0.2	RDT	1451	DATA Seq=3

Figura 5: Pedido de DOWNLOAD y envío de ACK inicial

A diferencia de los enfoques secuenciales estrictos, este escenario exige validar la correcta gestión de una ventana deslizante de transmisión, donde el servidor (10.0.0.1) transmite ráfagas de segmentos (DATA), delegando en el cliente (10.0.0.2) la responsabilidad de gestionar la recibida de cada paquete y emitir acuses de recibo individuales

609	0.571943549	10.0.0.2	10.0.0.1	RDT	51	ACK Seq=141
610	0.674381940	10.0.0.1	10.0.0.2	RDT	1451	DATA Seq=139
611	0.674395624	10.0.0.1	10.0.0.2	RDT	1451	DATA Seq=139
612	0.674506222	10.0.0.2	10.0.0.1	RDT	51	ACK Seq=139
613	0.674513336	10.0.0.2	10.0.0.1	RDT	51	ACK Seq=139
614	0.674653912	10.0.0.1	10.0.0.2	RDT	51	CLOSE
615	0.674659809	10.0.0.1	10.0.0.2	RDT	51	CLOSE
616	0.674771412	10.0.0.2	10.0.0.1	RDT	51	ACK Seq=142
617	0.674777688	10.0.0.2	10.0.0.1	RDT	51	ACK Seq=142
618	0.674798969	10.0.0.2	10.0.0.1	RDT	51	CLOSE
619	0.674800841	10.0.0.2	10.0.0.1	RDT	51	CLOSE
620	0.684891742	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=143
621	0.684906723	10.0.0.1	10.0.0.2	RDT	51	ACK Seq=143

Figura 6: Teardown en Download

Las capturas evidencian cómo el cliente procesa y almacena en búfer segmentos rezagados o fuera de orden mientras se inicia el proceso de desconexión. La observabilidad del intercambio iterativo de paquetes CLOSE y sus respectivos ACK demuestra que los temporizadores de retransmisión actúan correctamente, garantizando el cierre de conexión.

5. Preguntas a responder

1. Describa la arquitectura Cliente-Servidor.

Es una arquitectura de diseño de software distribuido donde los actores se dividen entre pro-

veedores de recursos (servidores) y demandantes de servicios (clientes), conectados por una red. En el inicio de la interacción, el servidor actúa como una entidad pasiva que permanece en estado de escucha a través de un puerto de red, a la espera de peticiones entrantes de los clientes. Cuando un cliente solicita un recurso específico, envía una petición al servidor utilizando el protocolo correspondiente. El servidor recibe la solicitud, la procesa y reúne la información necesaria para responder. Luego, envía una respuesta de vuelta al cliente, que la interpreta y la presenta al usuario por ejemplo. Es común encontrar esta arquitectura en servidores web y en servidores de bases de datos.

2. ¿Cuál es la función de un protocolo de capa de aplicación?

Un protocolo de capa de aplicación define un conjunto de reglas y estándares que permiten que las aplicaciones se comuniquen a través de una red. Funcionan definiendo los formatos de datos y los procedimientos para el intercambio de información entre diferentes sistemas. En particular, un protocolo de capa de aplicación define:

- Los tipos de mensajes intercambiados, por ejemplo, mensajes de solicitud y mensajes de respuesta.
- La sintaxis de los distintos tipos de mensajes, como los campos en el mensaje y cómo están delimitados dichos campos.
- La semántica de los campos, es decir, el significado de la información en los campos.
- Las reglas para determinar cuándo y cómo un proceso envía mensajes y responde a los mensajes.

Por ejemplo, HTTP, el protocolo de aplicación de la World Wide Web, define el formato y la secuencia de mensajes intercambiados entre nuestro navegador y el servidor web para poder cargar páginas web, imágenes y otros recursos.

3. Detalle el protocolo de aplicación desarrollado en este trabajo.

El protocolo de aplicación desarrollado es un protocolo de transferencia confiable de archivos (RDT o Reliable Data Transfer) sobre UDP que implementa dos operaciones principales: carga (upload) y descarga (download) de archivos. Está diseñado para funcionar sobre dos mecanismos de error-recovery: Stop & Wait y Selective Repeat. El protocolo implementa un modelo cliente-servidor con hilos workers dedicados por transferencia, asegurando que múltiples usuarios puedan transferir archivos concurrentemente sin interferencia. La arquitectura de dos capas (dispatcher + workers) permite escalabilidad y manejo de concurrencia de manera sencilla. El cliente puede elegir entre los dos mecanismos de error-recovery mencionados, mientras que el servidor se adapta transparentemente a la elección del cliente.

El protocolo implementa un sistema robusto de control de errores mediante el datagram ERROR (opcode 4), que transmite mensajes descriptivos de fallos al cliente. Las validaciones ocurren en múltiples capas: el dispatcher verifica nombre de archivo válido y tamaño máximo de archivo (20 MB), mientras que los workers detectan inactividad del remitente mediante timeouts de recepción (15 segundos). Para garantizar entrega confiable de errores, incluso en condiciones adversas, se utiliza la función *send_error_reliably()* que reintenta el envío hasta 25 veces. A nivel de concurrencia, el servidor dispatcher utiliza un lock (*threading.Lock*) para proteger la estructura *_active_transfers*, un diccionario que registra transferencias en curso identificadas por el par (dirección_cliente, nombre_archivo). Este mecanismo evita race conditions que podrían permitir múltiples workers procesando simultáneamente la misma transferencia, garantizando atomicidad en operaciones de lectura/escritura del estado compartido. Adicionalmente, el servidor implementa un garbage collector automático que limpia transferencias expiradas (antigüedad > 60 segundos) bajo protección del lock, previniendo memory leaks en sesiones largas con múltiples clientes concurrentes.

4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar

cada uno?

En cuanto a UDP, aparte de la función de multiplexación/demultiplexación y algunas comprobaciones de errores básicas (checksum), no añade nada a IP. De hecho, si el desarrollador de la aplicación elige UDP en lugar de TCP, la aplicación se comunica prácticamente de forma directa con IP. UDP recibe mensajes del proceso de la aplicación, adjunta los campos de número de puerto de origen y destino para el servicio de multiplexación/demultiplexación, añade otros dos campos mínimos y pasa el segmento resultante a la capa de red. Cuando el segmento llega al host receptor, UDP utiliza el número de puerto de destino para entregar los datos del segmento al proceso de aplicación correcto. Cabe destacar que con UDP no hay intercambio de información entre las entidades de la capa de transporte emisora y receptora antes de enviar un segmento. Por este motivo, se dice que UDP es sin conexión.

Características:

- Mayor control a nivel de aplicación sobre qué datos se envían y cuándo. Con UDP, en cuanto un proceso de aplicación pasa datos a UDP, este los empaqueta dentro de un segmento UDP y lo pasa inmediatamente a la capa de red. Es decir, no existe un control de congestión que realente los envíos en caso de congestión ni retransmisión de mensajes en caso de pérdida.
- No se requiere establecer una conexión. UDP transmite directamente sin ningún paso previo formal. Por lo tanto, UDP no introduce ningún retraso al establecer una comunicación.
- No garantiza la entrega, no ordena los paquetes, no filtra datagramas duplicados y no emite confirmaciones de recepción (ACK). Si un paquete se corrompe o se descarta en un enrutador intermedio, UDP no intenta recuperarlo.
- No existe estado de conexión, por lo que no se requiere el mantenimiento de buffers ni variables en los hosts para mantener el estado.
- Headers más livianos a diferencia de TCP (8 bytes vs 20 bytes)

Las aplicaciones en tiempo real, como la telefonía por Internet y las videoconferencias (VoIP), se ven muy afectadas por el control de congestión de TCP. Por estas razones, los desarrolladores de aplicaciones multimedia pueden optar por ejecutar sus aplicaciones sobre UDP en lugar de TCP. UDP es idóneo para sistemas donde la velocidad, el bajo delay y la reducción de overhead de control son críticos, y donde la aplicación puede tolerar la pérdida de datos o tiene sus propios mecanismos para lidiar con ella.

En cuanto a TCP, es un protocolo de transporte complejo diseñado para ofrecer una abstracción de un canal de comunicación confiable sobre una red IP inherentemente no confiable.

Características:

- Orientado a la conexión: Existe una serie de pasos definidos para el establecimiento de conexión (Three-way Handshake) para sincronizar estados entre el cliente y el servidor antes de transferir los datos útiles.
- Mediante un sistema de números de secuencia y acuses de recibo (ACKs), garantiza que los datos lleguen a la aplicación receptora sin errores, sin duplicados y en el orden secuencial exacto en que fueron emitidos (TCP en el receptor se encarga de ello), retransmitiendo cualquier segmento perdido.
- Implementa un mecanismo dinámico de ventana de recepción que viaja en el encabezado para garantizar que un emisor rápido no sature ni desborde los buffers de memoria de un receptor lento.
- TCP modera su tasa de transmisión basándose en la percepción de congestión en los enrutadores intermedios, reduciendo su velocidad cuando detecta pérdidas para evitar la constante retransmisión de paquetes.

TCP es utilizado en casos donde la integridad y la seguridad de recepción (confiabilidad de envío) son prioridad, y donde la latencia provocada por las retransmisiones es aceptable.

Es utilizado para la navegación web (HTTP/1.1 y HTTP/2), la transferencia de archivos estáticos (FTP) y el correo electrónico (SMTP/IMAP).

5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.

El protocolo desarrollado cuenta parcialmente con mecanismos de control de congestión, específicamente en la variante de Selective Repeat. Mientras que Stop & Wait carece de estos mecanismos al no utilizar una ventana deslizante de transmisión, Selective Repeat implementa un algoritmo AIMD (Additive Increase, Multiplicative Decrease) que adapta dinámicamente el tamaño de la ventana de congestión (cwnd) según las condiciones de la red. El mecanismo opera con un incremento aditivo que expande la ventana en 1 unidad por cada cantidad de ACKs consecutivos recibidos igual al tamaño actual de la ventana, permitiendo un crecimiento controlado cuando el canal está sin congestión. Ante la detección de pérdidas de paquetes (identificadas por timeouts), el protocolo aplica un decremento multiplicativo que reduce la ventana a la mitad, proporcionando una reacción rápida y agresiva a situaciones de congestión. La ventana mantiene límites configurables entre 5 y 500 paquetes, asegurando un comportamiento predecible y evitando tanto la subutilización como el envío excesivo de datos (congestionando la red).

6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?

El protocolo Selective Repeat logra una mayor eficiencia temporal en comparación con Stop & Wait, especialmente en archivos de gran volumen y redes con alta latencia. Esta superioridad radica en la técnica de pipelining, que permite el envío de múltiples datagramas de forma consecutiva sin necesidad de esperar la confirmación individual de cada uno, maximizando así el aprovechamiento del ancho de banda disponible. Mientras que en Stop & Wait el canal permanece ocioso durante el tiempo de ida y vuelta (RTT) de cada paquete, Selective Repeat mantiene un flujo constante de datos. No obstante, la elección no es absoluta: para archivos pequeños o redes con una tasa de pérdida despreciable, la simplicidad de Stop & Wait minimiza la sobrecarga computacional, mientras que Selective Repeat se consolida como la opción óptima para transferencias pesadas gracias a su capacidad de retransmisión selectiva, evitando el reenvío innecesario de datos correctamente recibidos.

6. Dificultades encontradas

Durante el desarrollo del proyecto, la principal dificultad técnica residió en el diseño y la implementación de un protocolo de capa de aplicación robusto sobre UDP, lo que exigió una gestión manual de la fiabilidad que normalmente delega el protocolo TCP. Esto implicó desafíos críticos en la serialización y empaquetado de datagramas, garantizando que estructuras complejas (como encabezados fijos y payloads variables) fueran interpretadas de forma idéntica por entidades independientes. Asimismo, la gestión de la concurrencia presentó una curva de aprendizaje importante, específicamente al coordinar la transición de estados entre el Dispatcher y los Workers mediante el uso de puertos efímeros y multihilo. Finalmente, la sincronización de la lógica de Stop & Wait ante condiciones adversas de red, como la pérdida de paquetes o la recepción de ACKs duplicados, requirió un diseño riguroso de timeouts y mecanismos de retransmisión para asegurar la integridad de los archivos transferidos sin comprometer la estabilidad del servidor.

7. Conclusión

La realización de este proyecto permitió comprender que la confiabilidad en la transferencia de datos no es una garantía del medio físico, sino el resultado de un software bien diseñado. Al trabajar con UDP, nos vimos obligados a asumir en la capa de aplicación responsabilidades que

normalmente delegamos en TCP, como el control de errores, el reordenamiento de paquetes y la gestión de retransmisiones.

La implementación de Stop & Wait y Selective Repeat fue clave para evidenciar cómo la latencia afecta el rendimiento y cómo técnicas como el pipelining pueden optimizar el flujo de datos. Por otro lado, la estructura de Dispatcher y Workers demostró ser la solución eficiente para gestionar la concurrencia, permitiendo que el servidor atienda múltiples clientes en paralelo sin interrumpir su escucha principal. En conclusión, el desarrollo del TP deja claro que, cuando el protocolo de transporte no garantiza la integridad, la lógica de la aplicación debe ser lo suficientemente robusta para sincronizar a emisor y receptor, asegurando una transferencia de archivos exitosa en cualquier condición de red.